# Stochastic Simulation Coursework 2023(1)

December 7, 2023

## 1 Stochastic Simulation - Coursework 2023

This assignment has two parts and graded over **100 marks**. Some general remarks:

- The assignment is due on **11 December 2023, 1PM GMT**, to be submitted via Blackboard (see the instructions on the course website).
- You should use this .ipynb file as a skeleton and you should submit a PDF report. Prepare the IPython notebook and export it as a PDF. If you can't export your notebook as PDF, then you can export it as HTML and then use the "Print" feature in browser (Chrome: File -> Print) and choose "Save as PDF".
- Your PDF should be no longer than 20 pages. But please be concise.
- You can reuse the code from the course material but note that this coursework also requires novel implementations. Try to personalise your code in order to avoid having problems with plagiarism checks. You can use Python's functions for sampling random variables of all distributions of your choice.
- **Please comment your code properly.**

Let us start our code.

[1]:
```
import matplotlib.pyplot as plt
import numpy as np

rng = np.random.default_rng(36) # You can change this.
```

### 1.1 Q1: Model Selection via Perfect Monte Carlo (40 marks)

Consider the following probabilistic model

$$p(x) = \mathcal{N}(x; 5, 0.01),$$
$$p(y_i|x) = \mathcal{N}(y_i; \theta x, 0.05),$$

for $i = 1, \ldots, T$ where $y_i$ are conditionally independent given $x$. You are given a dataset (see it on Blackboard) denoted here as $y_{1:T}$ for $T = 100$. As defined in the course, we can find the marginal likelihood as

$$p_\theta(y_{1:T}) = \int p_\theta(y_{1:T}|x)p(x)\mathrm{d}x,$$

where we have left $\theta$-dependence in the notation to emphasise that the marginal likelihood is a function of $\theta$.

Given the samples from prior $p(x)$, we can identify our test function above as $\varphi(x) = p_\theta(y_{1:T}|x)$.

**(i)** The first step is to write a log-likelihood function of $y_{1:T}$, i.e., $p_\theta(y_{1:T}|x)$. Note that, this is the joint likelihood of conditionally i.i.d. observations $y_i$ given $x$. This function should take input the data set vector `y` as loaded from `y_perfect_mc.txt` below, $\theta$ (scalar), and $x$ (scalar), and `sig` (likelihood variance which is given as 0.05 in the question but leave it as a variable) values to evaluate the log-likelihood. Note that log-likelihood will be a **sum** in this case, over individual log-likelihoods. **(10 marks)**

[2]:
```
# put the dataset in the same folder as this notebook
# the following line loads y_perfect_mc.txt
y = np.loadtxt('y_perfect_mc.txt')
y = np.array(y, dtype=np.float64)

# fill in your function below.

def log_likelihood(y, x, theta, sig): # fill in the arguments
    # fill in the function

    # Calculate the log-likelihood for each observation
    log_likelihoods = -0.5 * np.log(2 * np.pi * sig) - 0.5 * ((y - theta *␣
 ↪x)**2) / (sig)

    # Since the y's are independent conditional on x, we can sum up the␣
 ↪log-likelihoods
    # for each observation to get the total log-likelihood
    total_log_likelihood = np.sum(log_likelihoods)

    return total_log_likelihood

# uncomment and evaluate your likelihood (do not remove)
print(log_likelihood(y, 1, 1, 1))
print(log_likelihood(y, 1, 1, 0.1))
print(log_likelihood(y, -1, 2, 0.5))
```

```
-9898.905478066723
-98046.88084613332
-28974.21408410412
```

**(ii)** Write a logsumexp function. Let $\mathbf{v}$ be a vector of log-quantities and assume we need to compute $\log \sum_{i=1}^{N} \exp(v_i)$ where $\mathbf{v} = (v_1, \ldots, v_N)$. This function is given as

$$\log \sum_{i=1}^{N} \exp(v_i) = \log \sum_{i=1}^{N} \exp(v_i - v_{\max}) + v_{\max},$$

where $v_{\max} = \max_{i=1,\ldots,N} v_i$. Implement this as a function which takes a vector of log-values and returns the log of the sum of exponentials of the input values. **(10 marks)**

[3]:
```
def logsumexp(v):
    # v is a vector
    # fill in the function

    # Find the maximum value in the input vector v
    vmax = np.max(v)

    # Compute the log-sum of exponentials
    log_sum_exp = np.log(np.sum(np.exp(v - vmax))) + vmax

    return log_sum_exp

# uncomment and evaluate your logsumexp function (do not remove)
print(logsumexp(np.array([1, 2, 3])))
print(logsumexp(np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])))
print(logsumexp(np.array([5, 6, 9, 12])))
```

```
3.4076059644443806
10.45862974442671
12.051811977232925
```

**(iii)** Now we are at the stage of implementing the log marginal likelihood estimator. Inspect your estimator as described in Part (i). Its particular form is not implementable without using the trick you have coded in Part (iii). Now, implement a function that returns the **log** of the MC estimator you derived in Part (i). This function will take in

- `y` dataset vector
- $\theta$ parameter (scalar)
- `x_samples` (`np.array` vector) which are $N$ Monte Carlo samples.
- a variance (scalar) variable `sig` for the joint log likelihood $p_\theta(y_{1:T}|x)$ that will be used in `log_likelihood` function (we will set this to 0.05 as given in the question).

**Hint:** Notice that the log of the MC estimator of the marginal likelihood takes the form

$$\log \frac{1}{N} \sum_{i=1}^{N} p_\theta(y_{1:T}|x^{(i)}),$$

as given in the question. You have to use $p_\theta(y_{1:T}|x^{(i)}) = \exp(\log p_\theta(y_{1:T}|x^{(i)}))$ to get a `logsumexp` structure, i.e., log and sum (over particles) and exp of $\log p_\theta(y_{1:T}|x^{(i)})$ where $i = 1, \dots, N$ and $x^{(i)}$ are the $N$ Monte Carlo samples (do **not** forget $1/N$ term too). Therefore, now use the function of Part (i) to compute $\log p_\theta(y_{1:T}|x^{(i)})$ for every $i = 1, \dots, N$ and Part (ii) `logsumexp` these values to compute the estimate of log marginal likelihood. **(10 marks)**

[4]:
```
def log_marginal_likelihood(y, theta, x_samples, sig): # fill in the arguments
    # fill in the function

    # Compute log likelihood for each sample
    log_likelihoods = [log_likelihood(y, x, theta, sig) for x in x_samples]

    # Apply logsumexp to the computed log likelihoods
```

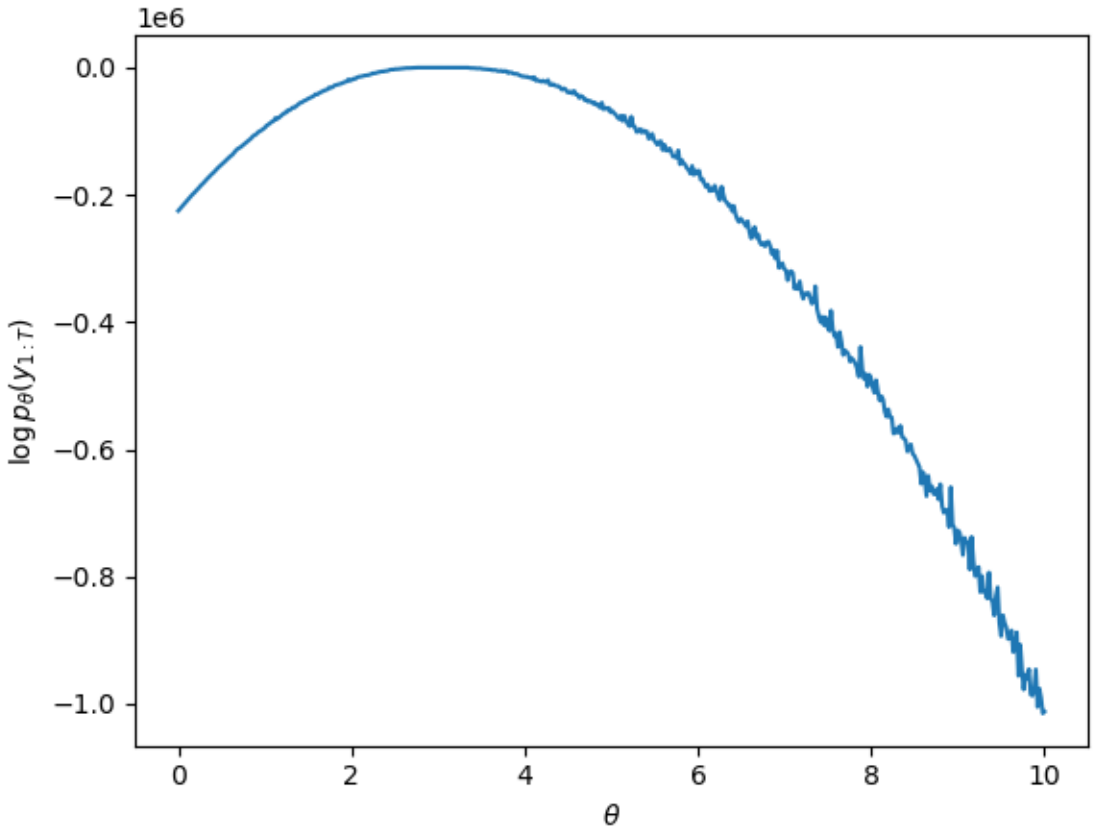
```
    logsumexp_result = logsumexp(np.array(log_likelihoods))

    # Divide by the number of samples and return
    log_marginal_likelihood = logsumexp_result - np.log(len(x_samples))

    return log_marginal_likelihood

# uncomment and evaluate your marginal likelihood (do not remove)

print(log_marginal_likelihood(y, 1, np.array([-1, 1]), 1))
print(log_marginal_likelihood(y, 1, np.array([-1, 1]), 0.1))
print(log_marginal_likelihood(y, 2, np.array([-1, 1]), 0.5))

# note that the above test code takes 2 dimensional array instead of N particles for simplicity
```

```
-9899.598625247283
-98047.57399331388
-16970.96811085916
```

**(iv)** We will now try to find the most likely $\theta$. For this part, you will run your `log_marginal_likelihood` function for a range of $\theta$ values. Note that, for every $\theta$ value, you need to sample $N$ new samples from the prior (do not reuse the same samples). Compute your estimator of the $\log \hat{\pi}^N_{\mathrm{MC}} \approx \log p_\theta(y_{1:T})$ for $\theta$-range given below. Plot the log marginal likelihood estimator as a function of $\theta$. **(5 marks)**

[5]:
```
sig = 0.05
sig_prior = 0.01
mu_prior = 5.0

N = 1000

theta_range = np.linspace(0, 10, 500)
log_ml_list = np.array([]) # you can use np.append to add elements to this array

# Iterate over theta_range
for theta in theta_range:
    # Sample N new samples from the prior for each theta
    x_samples = rng.normal(mu_prior, np.sqrt(sig_prior), N)

    # Compute log marginal likelihood for each theta
    log_ml = log_marginal_likelihood(y, theta, x_samples, sig)

    # Append the computed log_ml to log_ml_list
    log_ml_list = np.append(log_ml_list, log_ml)

# fill in your code here
```

```
# uncomment and plot your results (do not remove)

plt.plot(theta_range, log_ml_list)
plt.xlabel(r'$\theta$')
plt.ylabel(r'$\log p_\theta(y_{1:T})$')
plt.show()
```




**(v)** Now you have `log_ml_list` variable that corresponds to marginal likelihood values in `theta_range`. Find the $\theta$ value that gives the maximum value in this list and provide your final estimate of most likely $\theta$. **(5 marks)**

[6]:
```
# You code goes here

# Find the index of the maximum value in log_ml_list
max_index = np.argmax(log_ml_list)

# Find the corresponding theta value
theta_est = theta_range[max_index]

# print your theta estimate, e.g.:
```

```
print(theta_est)
```

```
2.9859719438877756
```

### 1.2 Q2: Posterior sampling (35 marks)

In this question, we will perform posterior sampling for the following model

$$p(x) \propto \exp(-x_1^2/10 - x_2^2/10 - 2(x_2 - x_1^2)^2),$$
$$p(y|x) = \mathcal{N}(y; Hx, 0.1)$$

where $H = [0, 1]$. In this exercise, we assume that we have observed $y = 2$ and would like to implement a few sampling methods.

Before starting this exercise, please try to understand how the posterior density should look like. The discussion we had during the lecture about Exercise 6.2 (see Panopto if you have not attended) should help you here to understand the posterior density. Note though quantities and various details are **different** here. You should have a good idea about the posterior density before starting this exercise to be able to set the hyperparameters such as the chain-length, proposal noise, and the step-size.

[7]:
```
y = np.array([2.0])
sig_lik = 0.1
H = np.array([0, 1])
```

**(i)** In what follows, you will have to code the log-prior and log-likelihood functions. Do **not** use any library, code the log densities directly. **(5 marks)**

[8]:
```
def prior(x):  # code banana density for visualisation purposes
    # fill in the function

    # Compute the prior.
    prior = np.exp(-x[0] ** 2 / 10 - x[1] ** 2 / 10 - 2 * (x[1] - x[0] ** 2) **␣
 ↪2)

    return prior


def log_prior(x):  # fill in the arguments
    # fill in the function

    log_prior = -x[0] ** 2 / 10 - x[1] ** 2 / 10 - 2 * (x[1] - x[0] ** 2) ** 2

    return log_prior


def log_likelihood(y, x, sig_lik):  # fill in the arguments
    # fill in the function

    # Compute the mean of the likelihood
```

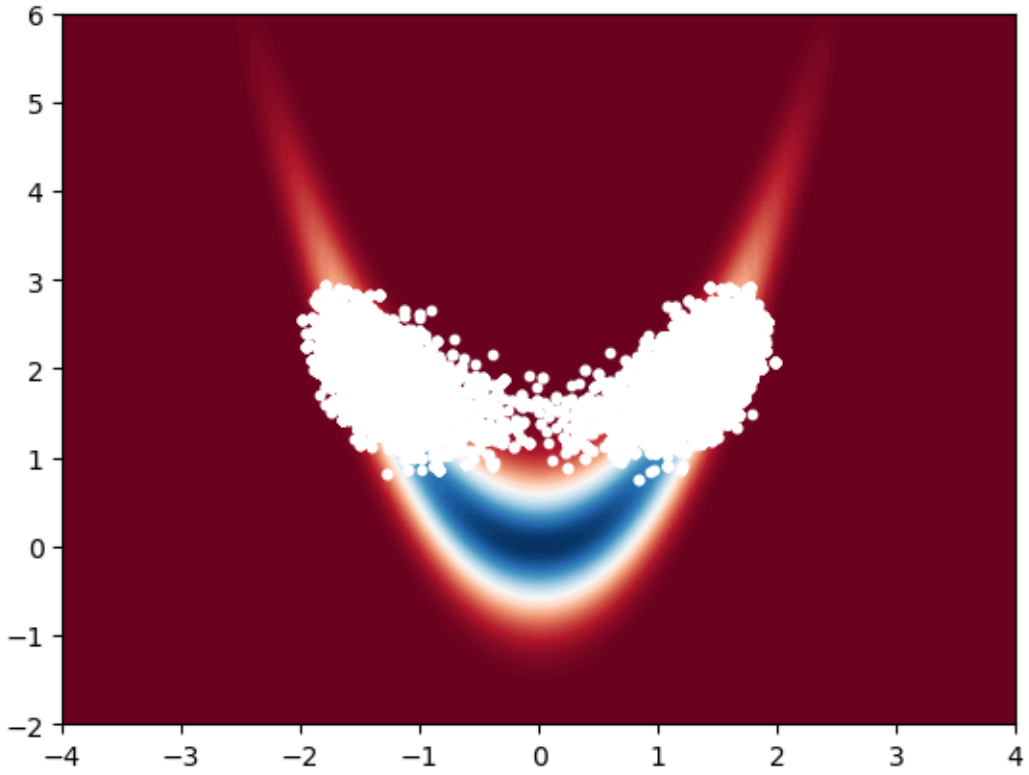
```
    mu = np.dot(H, x)

    # Compute the log-likelihood
    log_likelihood = -0.5 * np.log(2 * np.pi * sig_lik) - 0.5 * ((y - mu) ** 2)␣
 ↪/ (
        sig_lik
    )

    return log_likelihood


def log_posterior(y, x, sig_lik):
    return log_prior(x) + log_likelihood(y, x, sig_lik)


# uncomment below and evaluate your prior and likelihood (do not remove)
print(log_prior([0, 1]))
print(log_likelihood(y, np.array([0, 1]), sig_lik))
```

```
-2.1
[-4.76764599]
```

**(ii)** Next, implement **the random walk Metropolis algorithm (RWMH)** for this target. Set an appropriate chain length, proposal variance, and `burnin` value. Plot a scatter-plot with your samples (see the visualisation function below). Use log-densities only. **(10 marks)**

[9]:
```
# fill in your code here

proposal_std = 1  # Proposal standard deviation for the Gaussian proposal␣
 ↪distribution
chain_length = 50000  # Total length of the Markov Chain
burnin = 1000  # Burn-in period

x_current = np.random.randn(2)  # Initial point
x_current = np.array([0, 0])  # Initial point

samples = []
for _ in range(chain_length):
    x_proposal = x_current + np.random.normal(0, proposal_std, size=2)
    log_acceptance_ratio = log_posterior(y, x_proposal, sig_lik) -␣
 ↪log_posterior(y, x_current, sig_lik)

    if np.log(np.random.rand()) < log_acceptance_ratio:
        x_current = x_proposal

    samples.append(x_current)

x = np.array(samples[burnin:])
```

```
# uncomment and plot your results (do not remove)

x_bb = np.linspace(-4, 4, 100)
y_bb = np.linspace(-2, 6, 100)
X_bb, Y_bb = np.meshgrid(x_bb, y_bb)
Z_bb = np.zeros((100, 100))
for i in range(100):
    for j in range(100):
        Z_bb[i, j] = prior([X_bb[i, j], Y_bb[i, j]])
plt.contourf(X_bb, Y_bb, Z_bb, 100, cmap="RdBu")
plt.scatter(x[:, 0], x[:, 1], s=10, c="white")
plt.show()

# Note that above x vector (your samples) is assumed to be (N, 2).
# It does not have to be this way (You can change the name of the variable x␣
 ↪too).
# i.e., If your x vector is (2, N), then use
# plt.scatter(x[0, :], x[1, :], s=10 , c='white')
# instead of
# plt.scatter(x[:, 0], x[:, 1], s=10 , c='white')
# in the above code.
```

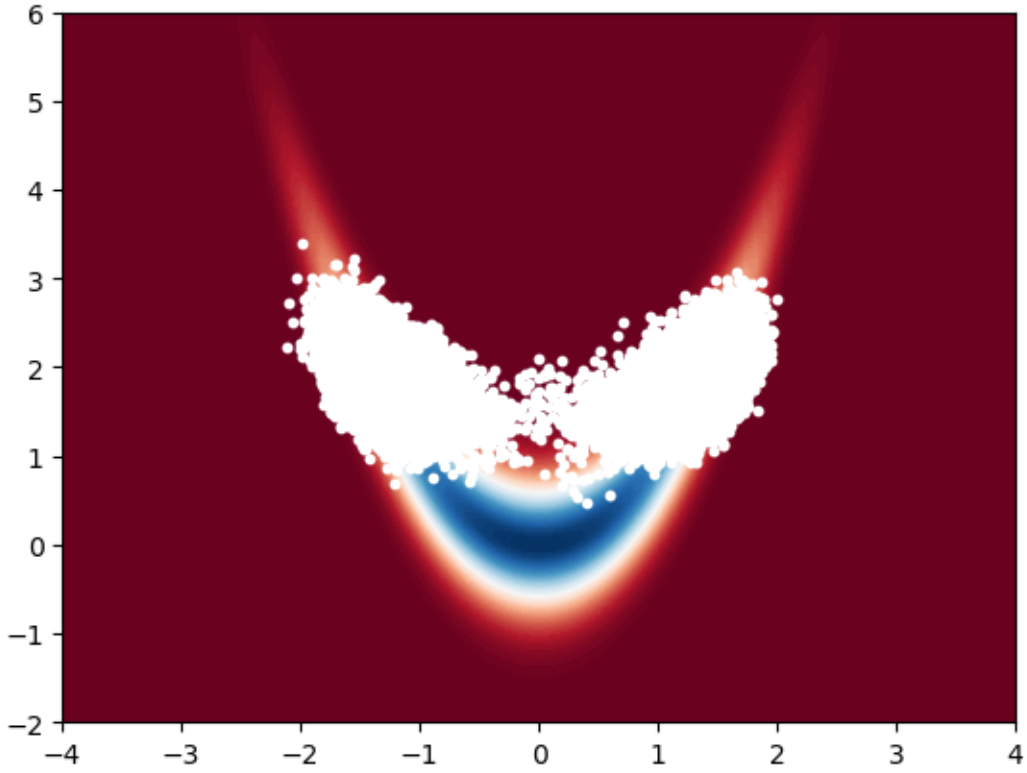


**(iii)** Now implement **Metropolis-adjusted Langevin algorithm**. For this, you will need to code the gradient of the density and use it in the proposal as described in the lecture notes. Set an appropriate chain length, step-size, and `burnin` value. Plot a scatter-plot with your samples (see the visualisation function below). Use log-densities only. **(10 marks)**

[10]:
```
def grad_log_prior(x):  # fill in the arguments
    # fill in the function

    grad_x0 = -x[0] / 5 + 8 * x[0] * (x[1] - x[0] ** 2)
    grad_x1 = -x[1] / 5 - 4 * (x[1] - x[0] ** 2)
    grad = np.array([grad_x0, grad_x1])

    return grad


def grad_log_likelihood(y, x, sig_lik):  # fill in the arguments
    # fill in the function

    grad = 1 / sig_lik * (y - np.dot(H, x)) * H

    return grad


def grad_log_posterior(y, x, sig_lik):
    return grad_log_prior(x) + grad_log_likelihood(y, x, sig_lik)


def log_MALA_kernel(y, x_proposal, x_current, gamma):  # fill in the arguments
    # fill in the function

    # Compute the parameters of the MALA kernel
    mu = x_current + gamma * grad_log_posterior(y, x_current, sig_lik)

    # Compute the log of the density of the MALA kernel
    log_pdf = - 1/ (4 * gamma) * np.linalg.norm(x_proposal - mu) ** 2

    return log_pdf


gam = 0.01

# fill in your code here

chain_length = 50000  # Total length of the Markov Chain
burnin = 1000  # Burn-in period
```

```
x_current = np.random.randn(2)  # Initial point
samples = []

for _ in range(chain_length):
    # Sample a proposal
    grad_log_density = grad_log_posterior(y, x_current, sig_lik)
    x_proposal = (
        x_current + gam * grad_log_density + np.sqrt(2 * gam) * np.random.
 ↪randn(2)
    )

    # Compute log of acceptance ratio
    log_acceptance_ratio = (
        log_posterior(y, x_proposal, sig_lik)
        + log_MALA_kernel(y, x_current, x_proposal, gam)
        - log_posterior(y, x_current, sig_lik)
        - log_MALA_kernel(y, x_proposal, x_current, gam)
    )

    # Accept or reject the proposal
    if np.log(np.random.rand()) < log_acceptance_ratio:
        x_current = x_proposal

    samples.append(x_current)

x = np.array(samples[burnin:])

# uncomment and plot your results (do not remove)
x_bb = np.linspace(-4, 4, 100)
y_bb = np.linspace(-2, 6, 100)
X_bb, Y_bb = np.meshgrid(x_bb, y_bb)
Z_bb = np.zeros((100, 100))
for i in range(100):
    for j in range(100):
        Z_bb[i, j] = prior([X_bb[i, j], Y_bb[i, j]])
plt.contourf(X_bb, Y_bb, Z_bb, 100, cmap="RdBu")
plt.scatter(x[:, 0], x[:, 1], s=10, c="white")
plt.show()

# Note that above x vector (your samples) is assumed to be (N, 2).
# It does not have to be this way (You can change the name of the variable x␣
 ↪too).
# i.e., If your x vector is (2, N), then use
# plt.scatter(x[0, :], x[1, :], s=10 , c='white')
# instead of
# plt.scatter(x[:, 0], x[:, 1], s=10 , c='white')
```

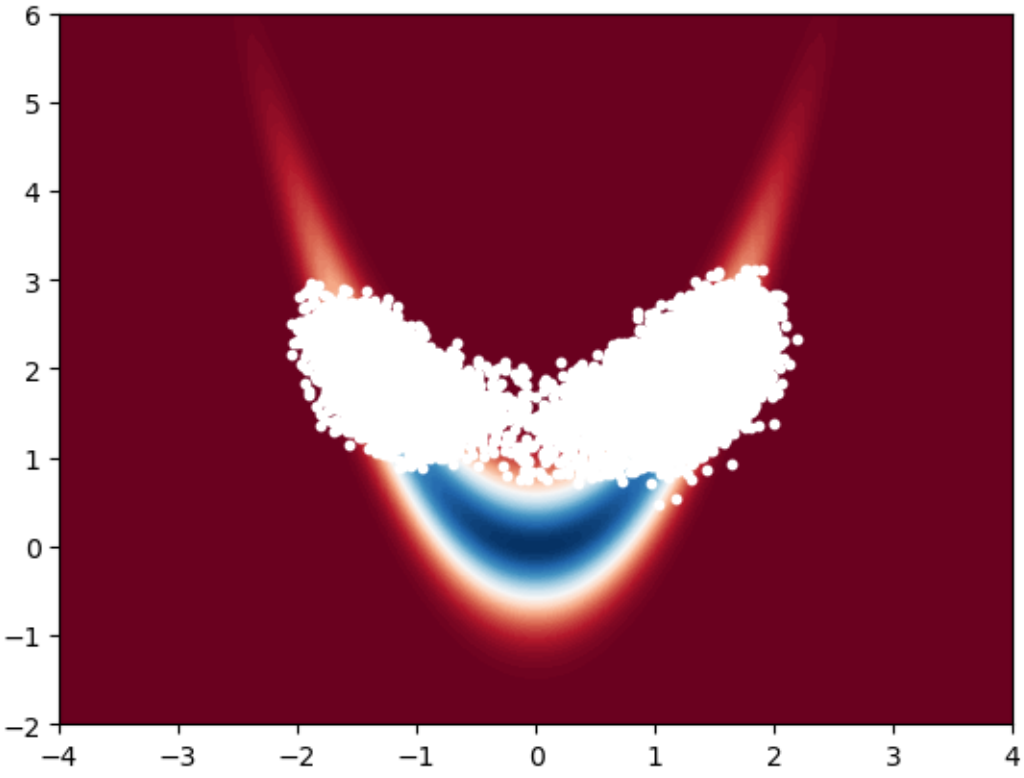
```
# in the above code.
```




**(iv)** Next, implement **unadjusted Langevin algorithm**. For this, you will need to code the gradient of the density and use it in the proposal as described in the lecture notes. Set an appropriate chain length, step-size, and `burnin` value. Plot a scatter-plot with your samples (see the visualisation function below). Use log-densities only. **(10 marks)**

[11]:
```
# fill in your code here

gam = 0.01

chain_length = 50000  # Total length of the Markov Chain
burnin = 1000  # Burn-in period

x_current = np.random.randn(2)  # Initial point
samples = []

for _ in range(chain_length):
    # Sample a proposal
    grad_log_density = grad_log_posterior(y, x_current, sig_lik)
    x_proposal = (
```

```
        x_current + gam * grad_log_density + np.sqrt(2 * gam) * np.random.
 ↪randn(2)
    )

    # Update the current point.
    x_current = x_proposal

    samples.append(x_current)

x = np.array(samples[burnin:])

# uncomment and plot your results (do not remove)
x_bb = np.linspace(-4, 4, 100)
y_bb = np.linspace(-2, 6, 100)
X_bb , Y_bb = np.meshgrid(x_bb , y_bb)
Z_bb = np.zeros((100 , 100))
for i in range(100):
    for j in range(100):
        Z_bb[i, j] = prior([X_bb[i, j], Y_bb[i, j]])
plt.contourf(X_bb , Y_bb , Z_bb , 100 , cmap='RdBu')
plt.scatter(x[:, 0], x[:, 1], s=10 , c='white')
plt.show()

# Note that above x vector (your samples) is assumed to be (N, 2).
# It does not have to be this way (You can change the name of the variable x␣
 ↪too).
# i.e., If your x vector is (2, N), then use
# plt.scatter(x[0, :], x[1, :], s=10 , c='white')
# instead of
# plt.scatter(x[:, 0], x[:, 1], s=10 , c='white')
# in the above code.
```




### 1.3 Q3: Gibbs sampling for 2D posterior (25 marks)

In this question, you will first derive a Gibbs sampler by deriving full conditionals. Then we will describe a method to estimate marginal likelihoods using Gibbs output (and you will be asked to implement the said method given the description).

Consider the following probabilistic model

$$\begin{aligned} p(x_1) &= \mathcal{N}(x_1; \mu_1, \sigma_1^2), \\ p(x_2) &= \mathcal{N}(x_2; \mu_2, \sigma_2^2), \\ p(y|x_1, x_2) &= \mathcal{N}(y; x_1 + x_2, \sigma_y^2), \end{aligned}$$

where $y$ is a scalar observation and $x_1, x_2$ are latent variables. This is a simple model where we observe a sum of two random variables and want to construct possible values of $x_1, x_2$ given the observation $y$.

**(i)** Derive the Gibbs sampler for this model, by deriving full conditionals $p(x_1|x_2, y)$ and $p(x_2|x_1, y)$ (You can use Example 3.2 but note that this case is different). **(10 marks)**

**Solution: Your answer here in LaTeX. If you prefer handwritten, please write here "Handwritten" and attach your pdf to the end and clearly number your answer Q3(i)**

$$
\begin{aligned}
p(x_1|x_2,y) = \frac{p(x_1,x_2,y)}{p(x_2,y)} &\propto p(x_1,x_2,y) = p(y|x_1,x_2)p(x_1,x_2) = p(y|x_1,x_2)p(x_1)p(x_2) \\
&\propto p(y|x_1,x_2)p(x_1) \\
&\propto \exp\left(-\frac{1}{2\sigma_y^2}(y-x_1-x_2)^2\right)\exp\left(-\frac{1}{2\sigma_1^2}(x_1-\mu_1)^2\right) \\
&\propto \exp\left(-\frac{1}{2\sigma_y^2}(y-x_1-x_2)^2-\frac{1}{2\sigma_1^2}(x_1-\mu_1)^2\right) \\
&\propto \exp\left(-\frac{1}{2\sigma_y^2}(y-x_1-x_2)^2-\frac{1}{2\sigma_1^2}(x_1^2-2x_1\mu_1+\mu_1^2)\right) \\
&\propto \exp\left(-\frac{1}{2\sigma_y^2}(y^2-2yx_1-2yx_2+x_1^2+2x_1x_2+x_2^2)-\frac{1}{2\sigma_1^2}(x_1^2-2x_1\mu_1+\mu_1^2)\right) \\
&\propto \exp\left(-\frac{1}{2\sigma_y^2}(x_1^2-2yx_1+2x_2x_1)-\frac{1}{2\sigma_1^2}(x_1^2-2x_1\mu_1)\right) \\
&\propto \exp\left\{-\frac{1}{2}\left\{\left(\frac{1}{\sigma_y^2}+\frac{1}{\sigma_1^2}\right)x_1^2-2\left(\frac{1}{\sigma_y^2}(y-x_2)+\frac{1}{\sigma_1^2}\mu_1\right)x_1\right\}\right\} \\
&\propto \exp\left\{-\frac{1}{2}\left\{\left(\sigma_y^{-2}+\sigma_1^{-2}\right)x_1^2-2\left(\sigma_y^{-2}(y-x_2)+\sigma_1^{-2}\mu_1\right)x_1\right\}\right\}
\end{aligned}
$$

Hence we have that:

$$
p(x_1|x_2,y) \sim \mathcal{N}\left(\frac{\sigma_y^{-2}(y-x_2)+\sigma_1^{-2}\mu_1}{\sigma_y^{-2}+\sigma_1^{-2}}, \frac{1}{\sigma_y^{-2}+\sigma_1^{-2}}\right)
$$

By symmetry, we also have that:

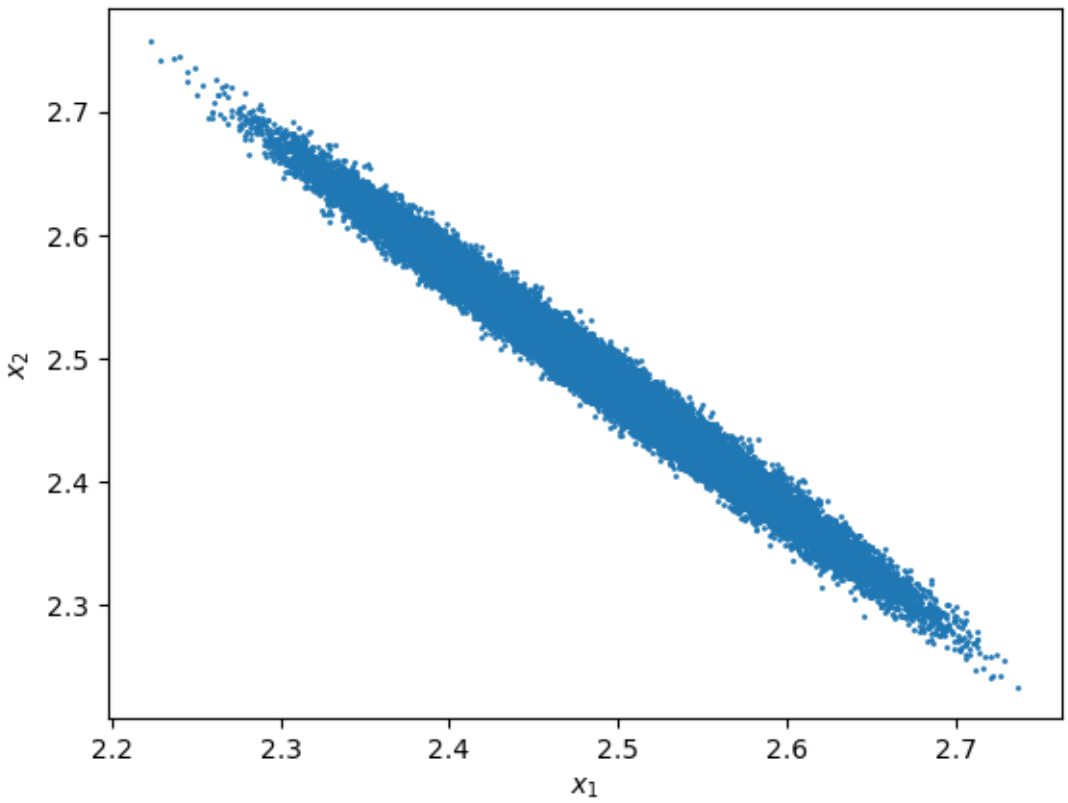
$$
p(x_2|x_1,y) \sim \mathcal{N}\left(\frac{\sigma_y^{-2}(y-x_1)+\sigma_2^{-2}\mu_2}{\sigma_y^{-2}+\sigma_2^{-2}}, \frac{1}{\sigma_y^{-2}+\sigma_2^{-2}}\right)
$$

**(ii)** Let us set $y = 5$, $\mu_1 = 0$, $\mu_2 = 0$, $\sigma_1 = 0.1$, $\sigma_2 = 0.1$, and $\sigma_y = 0.01$.

Implement the Gibbs sampler you derived in Part (i). Set an appropriate chain length and `burnin` value. Plot a scatter plot of your samples (see the visualisation function below). Discuss the result: Why does the posterior look like this? **(15 marks)**

[12]:
```
y = 5

mu1 = 0
mu2 = 0
sig1 = 0.1
sig2 = 0.1

sig_y = 0.01


# fill in your code here for Gibbs sampling
```

```python
chain_length = 50000  # Total length of the Markov Chain
burnin = 1000  # Burn-in period


# Standard deviation of the conditional distribution of x1
cond_std_x1 = np.sqrt(1 / (1 / (sig_y**2) + 1 / (sig1**2)))
# Standard deviation of the conditional distribution of x2
cond_std_x2 = np.sqrt(1 / (1 / (sig_y**2) + 1 / (sig2**2)))

x1_current = np.random.normal(mu1, sig1)  # Initial point
x2_current = np.random.normal(mu2, sig2)  # Initial point

x1_chain = [x1_current]
x2_chain = [x2_current]
for _ in range(chain_length):
    # Update the mean of the conditional distribution of x1
    cond_mean_x1 = (sig_y ** (-2) * (y - x2_current) + sig1 ** (-2) * mu1) / (
        sig_y ** (-2) + sig1 ** (-2)
    )
    # Sample x1 from the conditional distribution
    x1_current = np.random.normal(cond_mean_x1, cond_std_x1)

    # Update the mean of the conditional distribution of x2
    cond_mean_x2 = (sig_y ** (-2) * (y - x1_current) + sig2 ** (-2) * mu2) / (
        sig_y ** (-2) + sig2 ** (-2)
    )
    # Sample x2 from the conditional distribution
    x2_current = np.random.normal(cond_mean_x2, cond_std_x2)

    # Store the samples
    x1_chain.append(x1_current)
    x2_chain.append(x2_current)

x1_chain = np.array(x1_chain[burnin:])
x2_chain = np.array(x2_chain[burnin:])


# uncomment and plot your results (do not remove)
plt.scatter(x1_chain, x2_chain, s=1)
plt.xlabel(r"$x_1$")
plt.ylabel(r"$x_2$")
plt.show()
```




**Your discussion goes here. (in words).**

The scatter plot presents samples from the joint posterior distribution of $x_1$ and $x_2$ conditioned on $y$, that is, $p(x_1, x_2|y)$, obtained using our Gibbs sampler.

1. The plot looks like a straight line with negative slope. That is because the variables $x_1$ and $x_2$ are **negatively correlated** because their sum is close to 5. This is because $y$, which was observed to be equal to 5, is the sum of $x_1$ and $x_2$ plus some noise, which is much smaller than the observed value (since $\sigma_y = 0.01$).

2. There isn't to much variance on the line. That is because the variance of the conditional distributions $p(x_1|x_2, y)$ and $p(x_2|x_1, y)$ are not dependent on $x_1$ or $x_2$ and they are both equal to $\frac{1}{\sigma_y^{-2}+\sigma_1^{-2}} = \frac{1}{\sigma_y^{-2}+\sigma_2^{-2}} = \frac{1}{10^4+10^2}$, which is very small.